Analyzing Internal and External Parallelism in Multi-Index FAISS Retrieval: Threads, Processes, OpenMP and the Cost of Concurrency

A CPU Performance Engineering Study

Suwesh Prasad Sah
suwesh.github.io

August 2026

Abstract

This paper presents a performance-engineering study of a multi-index FAISS retrieval pipeline. Three independent retrieval sources were evaluated using sequential execution, thread-level parallelism through `ThreadPoolExecutor`, and process-level parallelism through `ProcessPoolExecutor`. Additional experiments were conducted using `OMP_NUM_THREADS=1` to isolate the contribution of FAISS internal parallelism from externally managed concurrency. Execution time measurements and profiling data were collected to understand the performance implications of different execution models. The results demonstrate the relationship between retrieval workload granularity, internal OpenMP parallelism, external concurrency mechanisms, and parallel execution overhead.

Keywords: Parallel Computing, Performance Engineering, Profiling, Concurrency, Workload Granularity
Artifacts and Supporting Evidence: Benchmark scripts, raw measurements, profiling statistics, and analysis artifacts are available here.
Notice: This manuscript is a technical report and has not undergone peer review.

---

1 Introduction

When multiple retrieval sources exist, independent searches can be executed concurrently using threads, processes, or internally parallel numerical libraries for each source. While such approaches are often expected to improve performance, the benefits depend on the relationship between the amount of useful computation and the overhead introduced by parallel execution.

This study investigates the performance characteristics of a multi-index FAISS retrieval pipeline consisting of three independent retrieval sources. The objective was not to maximize retrieval throughput, but to understand how different forms of parallel execution affect retrieval latency for a relatively small retrieval workload.

The evaluated retrieval system maintains three separate vector indexes representing distinct document collections. Due to differences in document structure and operational requirements, the collections are indexed separately and all indexes are queried for every incoming request. This architecture naturally presents an opportunity to evaluate alternative concurrency strategies for retrieval execution.

Three execution strategies were evaluated: sequential execution, thread-level parrallelism using `ThreadPoolExecutor`, and process-level parallelsim using `ProcessPoolExecutor`. To seperate the effects of FAISS internal OpenMP parallelism from externally managed concurrency, additional experiments were performed with `OMP_NUM_THREADS=1`.

The study seeks to answer the following research question:

> How do internal OpenMP parallelism, thread-level concurrency, and process-level concurrency influence the latency of multi-index FAISS retrieval workloads?

The primary contribution of this work is an empirical comparison of internal and external parallelization strategies using profiling runtime measurements obtained from a production-inspired retrieval pipeline.

2 Retrieval System Architecture

2.1 Retrieval Sources

The evaluated retrieval system maintains three independent vector indexes, each representing a distinct document collection within a broader enterprise knowledge ecosystem. The collections are indexed separately due to differences in document structure, ingestion requirements, and retrieval characteristics.

Operationally, every user query is evaluated against all three indexes. The resulting retrieved candidates are aggregated then reranked before being passed to subsequent stages.

2.2 Retrieval Pipeline

The production-inspired retrieval pipeline consists of four principal stages:

1. Query embedding generation.
2. Multi-index FAISS retrieval.
3. Candidate aggregation.
4. Result reranking.

Figure 1 illustrates the production-inspired retrieval pipeline evaluated throughout this study.

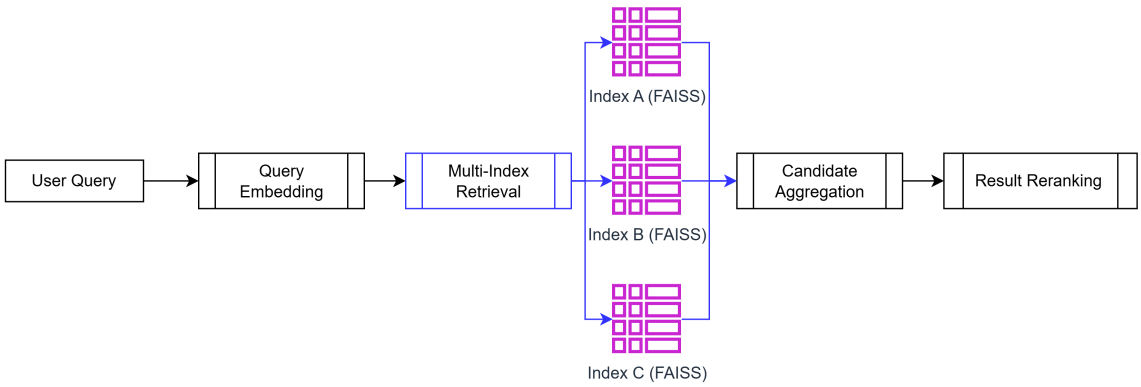

Figure 1: Execution-model agnostic retrieval pipeline. Retrieval latency measurements focused exclusively on the multi-index retrieval stage.

Although embedding generation and reranking are present in the complete system, they are not the focus of the present study. The experiments reported in this paper measure only the latency associated with the retrieval stage.

2.3 Retrieval Execution Model

At the multi-index FAISS retrieval stage, the query embedding is submitted to each index and a fixed number of nearest-neighbour candidates are retrieved using FAISS similarity search.

These retrieval operations are represented programmatically as a collection of independent search tasks:

```
search_tasks = [
    (search_index_a, (*args)),
    (search_index_b, (*args)),
    (search_index_c, (*args))
]
```

Depending on the execution model being evaluated, the three searches may be executed sequentially, concurrently using threads, or concurrently using separate processes.

3 Execution Models

Figure 2 illustrates the execution models evaluated in this study. The same retrieval workload was executed using sequential execution, thread-level parallelism, and process-level parallelism.

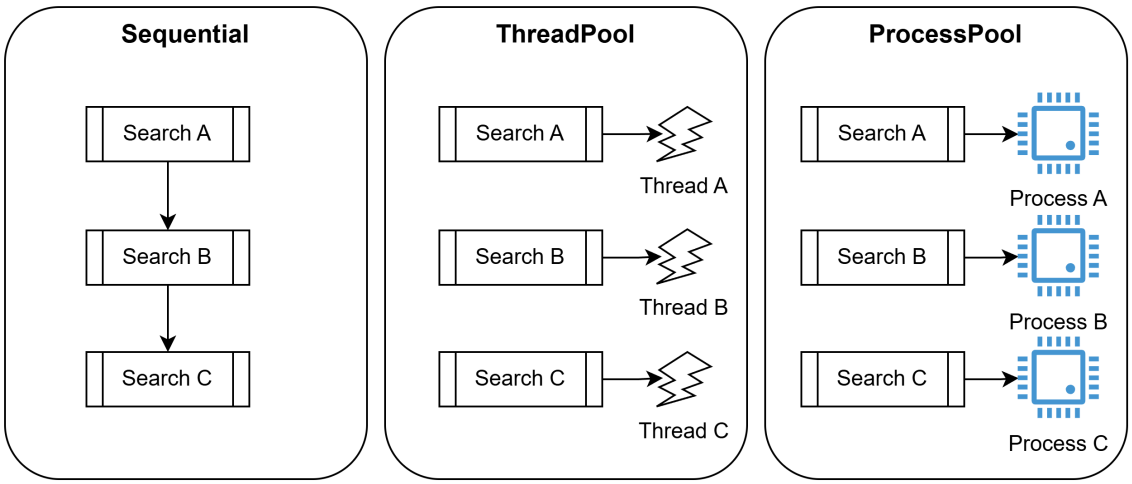

Figure 2: Execution models of the FAISS retrieval stage evaluated in this study.

3.1 Sequential Execution

The baseline implementation executes retrieval tasks sequentially. Each index is searched one after another using the same query embedding. Execution does not introduce any external concurrency and therefore serves as a reference configuration for subsequent comparisons.

```
for func, args in search_tasks:
    results.append(fun(*args))
```

3.2 Thread-Level Parallelism

The second execution model uses Python's `ThreadPoolExecutor` to dispatch retrieval tasks concurrently across multiple worker threads. Each retrieval task operates on an independent vector index while sharing the same process address space.

```
with ThreadPoolExecutor(max_workers=len(search_tasks)) as executor:
    futures = [
        executor.submit(func, *args)
        for func, args in search_tasks
    ]
```

3.3 Process-Level Parallelism

The third execution model uses `ProcessPoolExecutor` to execute retrieval tasks in separate operating system processes. Unlike the thread-level implementation, workers do not share the same address space and therefore require inter-process task dispatch and result transfer.

```
with ProcessPoolExecutor(max_workers=len(search_tasks)) as executor:
    futures = [
        executor.submit(func, *args)
        for func, args in search_tasks
    ]
```

4 Experimental Methodology

4.1 Hardware and Software Environment

All experiments were conducted on a workstation equipped with an AMD Ryzen 5 5500 processor (6 cores, 12 threads) and 32 GB DDR4 memory operating at 3200 MHz. The software environment consisted of Ubuntu 24.04.4 LTS running under Windows Subsystem for Linux 2 (WSL2) with Linux kernel version 6.6.87.2-microsoft-standard-WSL2. The retrieval system was implemented in Python 3.12 and utilized the CPU version of FAISS (`faiss-cpu==1.11.0.post1`) for vector similarity search. Consequently, all reported measurements reflect CPU retrieval performance.

4.2 Retrieval Workload

The evaluated retrieval workload consisted of a single query, denoted as Q, executed repeatedly against three independent FAISS indexes. All indexes and query embeddings used a dimensionality of 1024. The three retrieval indexes occupied approximately 256 KB, 565 KB and 106 KB respectively.

For all experiments, the query, retrieval indexes, search parameters, hardware environment, software environment, and retrieval logic remained unchanged. The only experimental variable was the execution model used to perform the three independent FAISS searches.

4.3 Measurement Procedure

Latency measurements were collected using Python's high-resolution timer `time.perf_counter()`.

Two measurement levels were recorded. First, end-to-end latency was measured externally around the complete invocation of the retrieval pipeline, including embedding generation, retrieval, and reranking stages. These measurements captured the total wall-clock latency observed by the caller. Second, instrumentation was added within the retrieval pipeline to isolate the latency of the multi-index FAISS retrieval stage for each execution model.

Each experimental configuration was executed ten times. The resulting measurements were used to compute descriptive statistics including mean, median, minimum, maximum, and standard deviation latency. Repeated execution was performed using the following command:

```
for i in {1..10}; do
    python benchmark_retrieval.py
done
```

Execution profiles were collected using Python's `cProfile` profiler and visualized using `tuna`. Profiling traces were generated separately from benchmark runs and were used for execution model analysis discussed later in the paper.

Although end-to-end measurements were recorded, the primary comparison in this study focuses on the latency of the FAISS retrieval stage for each execution model.

4.4 Controlled Runtime Configuration

FAISS uses OpenMP-based internal parallelism by default. Prior to the controlled experiments, the default runtime configuration reported a maximum OpenMP thread count of 12, corresponding to the available hardware threads on the experimental platform.

To separate application-level concurrency from FAISS internal parallelism, a second set of experiments was performed with OpenMP restricted to a single thread:

```
export OMP_NUM_THREADS=1
```

This configuration was verified using:

```
python -c "import faiss; print(faiss.omp_get_max_threads())"
```

which reported a value of 1 after applying the environment variable.

Without controlling OpenMP, thread-based and process-based execution models may interact with FAISS internal parallelism, creating nested parallelism and potentially introducing resource oversubscription. Restricting FAISS to a single OpenMP thread allows the effects of application-level execution models to be evaluated more directly.

All execution models (sequential, thread-level parallelism, and process-level parallelism) were evaluated under both the default OpenMP runtime configuration and the controlled `OMP_NUM_THREADS=1` configuration.

5 Performance Results

5.1 Default OpenMP Runtime Configuration

Table 1 summarizes the retrieval-stage latency measurements collected under the default FAISS OpenMP configuration.

Execution Model	Mean (ms)	Median (ms)	Min (ms)	Max (ms)	Std Dev (ms)
Sequential	0.3097	0.3050	0.2760	0.3880	0.0291
ThreadPoolExecutor	1.3572	1.3305	1.1150	1.5550	0.1540
ProcessPoolExecutor	69.4282	67.6035	64.0690	81.5160	5.1120

Table 1: FAISS retrieval-stage latency under the default OpenMP configuration.

5.1.1 Sequential Execution

Sequential execution produced the lowest measured latency among the evaluated configurations. The retrieval-stage latency remained below 0.4 ms across all runs.

5.1.2 Thread-Level Parallelism

ThreadPoolExecutor increased measured latency relative to sequential execution despite the independence of the retrieval tasks. Retrieval-stage latency remained approximately 1–2 ms across measured runs.

5.1.3 Process-Level Parallelism

ProcessPoolExecutor produced the highest retrieval-stage latency. Measured latency increased to approximately 65–80 ms, indicating that multiprocessing overhead exceeded the useful computational work performed by the retrieval workload.

5.2 Controlled OpenMP Runtime Configuration

Table 2 summarizes the retrieval-stage latency measurements collected with FAISS internal OpenMP parallelism restricted to a single thread (`OMP_NUM_THREADS=1`).

Execution Model	Mean (ms)	Median (ms)	Min (ms)	Max (ms)	Std Dev (ms)
Sequential	2.1413	0.8425	0.6640	7.9290	2.6506
ThreadPoolExecutor	1.4719	1.4330	1.2510	1.8090	0.1662
ProcessPoolExecutor	78.6354	69.6685	63.1530	160.2090	28.9809

Table 2: FAISS retrieval-stage latency under the controlled OpenMP configuration.

5.2.1 Sequential Execution

Under the controlled OpenMP configuration, sequential execution remained the lowest-latency execution model for most runs. Retrieval-stage latency was typically below 1 ms, although two measurements exhibited substantially higher latency, increasing the observed variance.

5.2.2 Thread-Level Parallelism

ThreadPoolExecutor produced retrieval-stage latencies comparable to the uncontrolled configuration. Retrieval latency remained approximately 1–2 ms across all measured runs with relatively low variability.

5.2.3 Process-Level Parallelism

ProcessPoolExecutor continued to exhibit the highest retrieval-stage latency after restricting FAISS to a single OpenMP thread. Retrieval latency remained approximately 60–80 ms for most runs, indicating that process-management overhead continued to dominate the retrieval workload.

6 Profiling Analysis

6.1 OpenMP Microbenchmark

To investigate FAISS internal parallelism independently from application-level concurrency, a microbenchmark consisting of 10,000 repeated searches against a single index was performed. The benchmark used the same query embedding and search parameters as the retrieval experiments presented in Section 5. A warm-up phase of 100 searches was performed prior to measurement. The benchmark was evaluated under both the default OpenMP configuration and a controlled `OMP_NUM_THREADS=1` configuration.

Maximum OMP Threads	Mean (μs)	Median (μs)	Min (μs)	Max (μs)	P95 (μs)	P99 (μs)
12	21.40	17.38	16.45	341.14	31.41	67.49
1	19.52	17.08	16.38	366.68	28.77	45.16

Table 3: Single-index FAISS search microbenchmark.

Only minor latency differences were observed between the default OpenMP configuration and `OMP_NUM_THREADS=1`. The median search latency remained approximately 17 μs in both cases, suggesting that the evaluated single-query workload was too small to significantly benefit from additional OpenMP threads.

These results provide additional context for the execution-model measurements presented in Section 5. While OpenMP configuration produced only microsecond-scale differences in individual search latency, thread- and process-based execution models introduced latency differences at the millisecond scale. This suggests that application-level orchestration overhead had a substantially larger impact on performance than FAISS internal parallelism for the evaluated workload.

6.2 Sequential Execution Profile

Figure 3 presents the Tuna visualizations obtained from the sequential retrieval implementation under both the default and controlled OpenMP configurations.

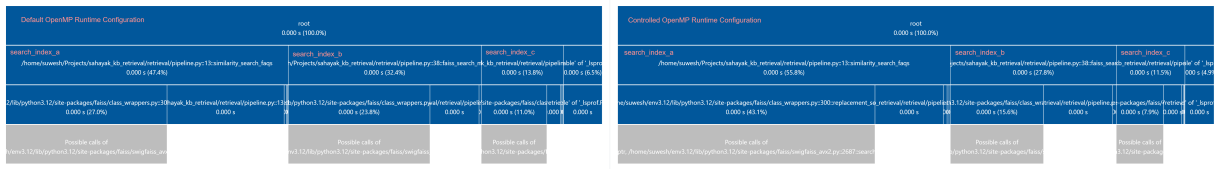

Figure 3: Sequential retrieval profiles under the default and controlled OpenMP configurations.

Across both configurations, the dominant execution path consisted of the retrieval-source search functions invoking FAISS's `replacement_search()` wrapper and the underlying `IndexFlat::search()` implementation. The overall profile structure remained largely unchanged after restricting FAISS to a single OpenMP thread.

These observations are consistent with the OpenMP microbenchmark results presented earlier in this section, which showed only minor latency differences between the default and controlled OpenMP configurations for the evaluated single-query workload.

6.3 Thread-Level Parallelism Profile

Figure 4 presents the Tuna visualizations obtained from the ThreadPoolExecutor implementation under both the default and controlled OpenMP configurations.

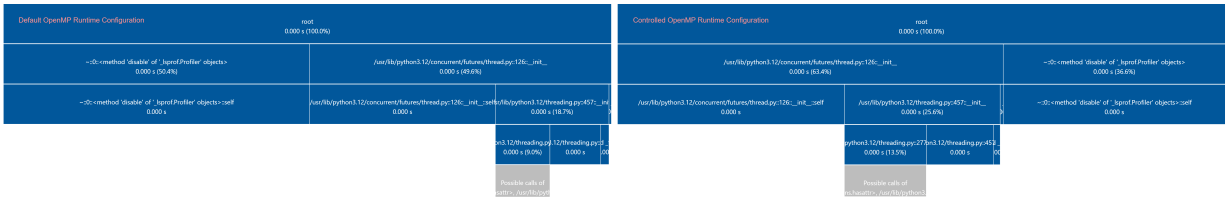

Figure 4: ThreadPoolExecutor retrieval profiles under the default and controlled OpenMP configurations.

Unlike the sequential profiles, the dominant execution paths were associated with the initialization and management of Python threading constructs. In both configurations, substantial portions of the profile were attributed to `ThreadPoolExecutor` initialization (`concurrent.futures.thread`) and the creation of worker thread objects (`threading.Thread`).

The appearance of thread-initialization functions in the profile is consistent with the increased retrieval latency reported in Section 5 relative to the sequential execution model.

6.4 Process-Level Parallelism Profile

Figure 5 presents the Tuna visualizations obtained from the ProcessPoolExecutor implementation under both the default and controlled OpenMP configurations.

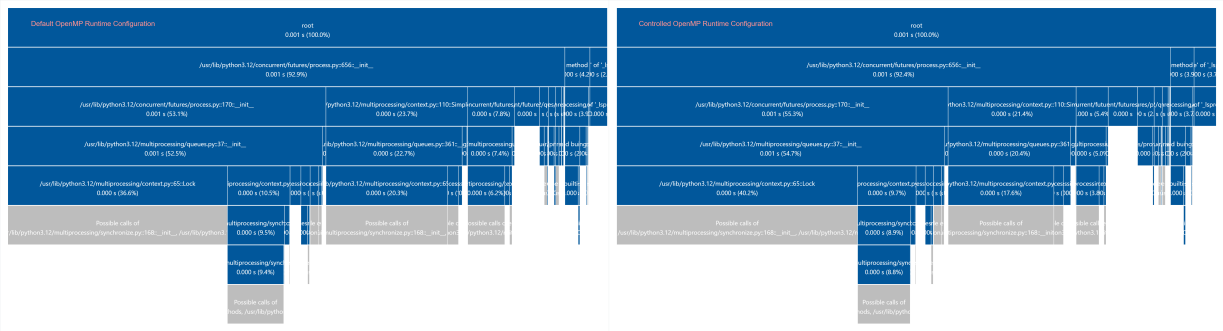

Figure 5: ProcessPoolExecutor retrieval profiles under the default and controlled OpenMP configurations.

Unlike both the sequential and thread-level profiles, the dominant execution paths were associated with Python's multiprocessing infrastructure. In both configurations, substantial portions of the profile were attributed to `ProcessPoolExecutor` initialization (`concurrent.futures.process`), multiprocessing queue management (`multiprocessing.queues`), synchronization primitives, and process-level locking mechanisms.

The prominence of process-management functions in the profile is consistent with the significantly higher retrieval-stage latency reported in Section 5 relative to both the sequential and ThreadPoolExecutor implementations.

7 Discussion

The experimental results demonstrate that the evaluated retrieval workload was highly sensitive to the chosen execution model. Sequential execution consistently produced the lowest retrieval-stage latency, while ProcessPoolExecutor produced the highest latency under both OpenMP configurations. Sequential execution was dominated by the FAISS retrieval path, whereas the ThreadPoolExecutor and ProcessPoolExecutor profiles increasingly exposed thread-management and process-management infrastructure.

The OpenMP microbenchmark showed that an isolated single-index search exhibited only minor latency differences between the default OpenMP configuration and `OMP_NUM_THREADS=1`. This suggests that the individual search operation itself was extremely fine-grained and that the substantial latency differences observed in Section 5 emerged primarily at the execution-model level rather than from the isolated search operation.

7.1 The Cost of Concurrency: Internal vs External Parallelism

An important distinction emerging from the experiments is the difference between internal and external parallelism. OpenMP introduces parallelism within the FAISS computation itself, whereas ThreadPoolExecutor and ProcessPoolExecutor introduce parallelism at the application layer by scheduling multiple retrieval operations concurrently.

Although the present measurements do not permit a quantitative decomposition of latency into process creation, serialization, queue management, synchronization, or inter-process communication costs, the profiling evidence indicates that executor-level orchestration overhead dominated

the evaluated workload. The experiment therefore highlights a broader performance-engineering principle: the effectiveness of parallelism depends not only on task independence, but also on workload granularity and the layer at which concurrency is introduced.

8 Threats to Validity

Several limitations should be considered when interpreting the reported results.

First, the evaluated workload was intentionally small, consisting of three FAISS indexes ranging from approximately 106 KB to 565 KB and a single query executed against all retrieval sources. Larger indexes, larger query batches, alternative search parameters, or different FAISS index families may exhibit different performance characteristics.

Second, the experiments were conducted on a single hardware and software configuration. Consequently, the results reflect the evaluated CPU platform, operating system environment, Python runtime, and FAISS implementation, and may not directly generalize to other systems.

Third, OpenMP behavior was controlled through `OMP_NUM_THREADS`, which specifies the maximum number of OpenMP threads available to FAISS. The study does not directly measure the number of threads utilized internally during each search operation.

Fourth, the OpenMP microbenchmark and the primary execution-model benchmark evaluate related but distinct scenarios. The microbenchmark examines repeated single-index searches in isolation, whereas the primary benchmark evaluates a complete three-source retrieval workflow.

Finally, the study focuses exclusively on retrieval latency. Retrieval quality, scalability under higher query volumes, GPU execution, alternative retrieval architectures, and larger-scale workloads are outside the scope of the present investigation.

9 Conclusion

This study investigated the effects of internal and external parallelism on a multi-index FAISS retrieval workload.

The results demonstrated that the location and granularity of parallelism influence whether its overhead can be amortized by useful computation. While the evaluated retrieval tasks were independent and therefore parallelizable, the computational work associated with each search was sufficiently small that externally managed thread- and process-level concurrency did not improve retrieval latency.

The experiment illustrates the central performance-engineering lesson behind the paper's title: concurrency is not a performance benefit in itself. Parallel execution introduces a cost, and the effectiveness of parallelism depends on where it is introduced and whether sufficient useful work exists to justify that cost.

Future work may investigate larger retrieval workloads, alternative FAISS index types, larger query batches, GPU-based retrieval, and higher-throughput scenarios in which the balance between useful computation and concurrency overhead may differ.